

BIU Deep Reinforcement Learning

Exercise 03

LSTM + REINFORCE + A2C for Personal Fitness Planning

Hodaya Kashkash
Bar-Ilan University — Dr. Yoram Segal

Abstract. This report presents a reinforcement-learning pipeline for personal workout planning. An LSTM transition model (val MSE = 0.0090) is trained on 448 days of real Hevy workout-log data to serve as a learned environment dynamics model. Two policy-gradient algorithms — REINFORCE and Advantage Actor-Critic (A2C) — are evaluated over 500 episodes each. A2C achieves a higher final average return (17.05 vs 15.34) and converges more reliably, consistent with its lower-variance TD advantage.

1. Dataset and Representation

The Hevy App Workout Dataset (Kaggle:

[tejaswinimukesh/hevy-app-workout-dataset-from-dumbbells-to-data](#)) provides 100 weeks of real exercise-log data covering 7,882 exercise-set records across 448 distinct workout days. Each row contains: exercise name, muscle group, sets index, reps, weight (lbs), and session timestamps.

Feature engineering. Raw exercise rows are aggregated to daily workout summaries. The state vector s_t (dim=5) is:

Index	Feature	Encoding
0	rolling_7day_load	MinMaxScaler $\rightarrow [0, 1]$
1	muscle_balance_score	Shannon entropy of muscle distribution
2	session_duration_avg	MinMaxScaler $\rightarrow [0, 1]$
3	day_sin	$\sin(2\pi * t / 28)$
4	day_cos	$\cos(2\pi * t / 28)$

Sinusoidal encoding for *day_in_cycle* is essential: a scalar fraction $t/27$ treats day 0 and day 27 as maximally distant, breaking the cyclical structure. sin/cos place day 0 and day 28 at the same point on the unit circle. The MinMaxScaler is fitted exclusively on the training split (80%) and applied without refitting to validation — preventing data leakage.

Action space. K-Means (k=6) clusters daily summaries into six workout archetypes: Rest, Upper Push, Upper Pull, Lower Body, Full Body, and Core/Cardio. This converts the continuous workout space into a discrete MDP action set.

2. LSTM Transition Model

The LSTM serves as a learned world model: $s_{t+1} = f(s_t, a_t, h_t)$. It is trained with supervised MSE loss on consecutive 7-day windows (351 train / 83 val), using Adam (lr=0.001) for 50 epochs.

Architecture. Embedding(6, 8) maps discrete actions to dense vectors, concatenated with the state at each timestep (input size = 13). Two LSTM layers (hidden=64, dropout=0.2) capture temporal dependencies; a linear head projects to state_dim=5.



Figure 1. LSTM training and validation MSE loss over 50 epochs. Train loss: 0.2913 → 0.0132. Val loss: 0.2717 → 0.0090.

Both curves decrease monotonically and converge to similar values, indicating the model generalises without overfitting. The validation loss slightly undercuts the training loss — consistent with dropout regularisation being active during training but disabled at evaluation time.

3. REINFORCE

REINFORCE trains a stochastic policy $\pi(a|s)$ using episodic Monte Carlo returns:

$$\theta \leftarrow \theta + \alpha * \sum_t [\text{grad} \log \pi(a_t | s_t) * (G_t - \text{mean}(G))]$$

Subtracting the episode mean $\text{mean}(G)$ as a baseline reduces gradient variance without introducing bias. Episodes are 28 days long; 500 episodes were trained with Adam ($\text{lr}=3\text{e-}4$).

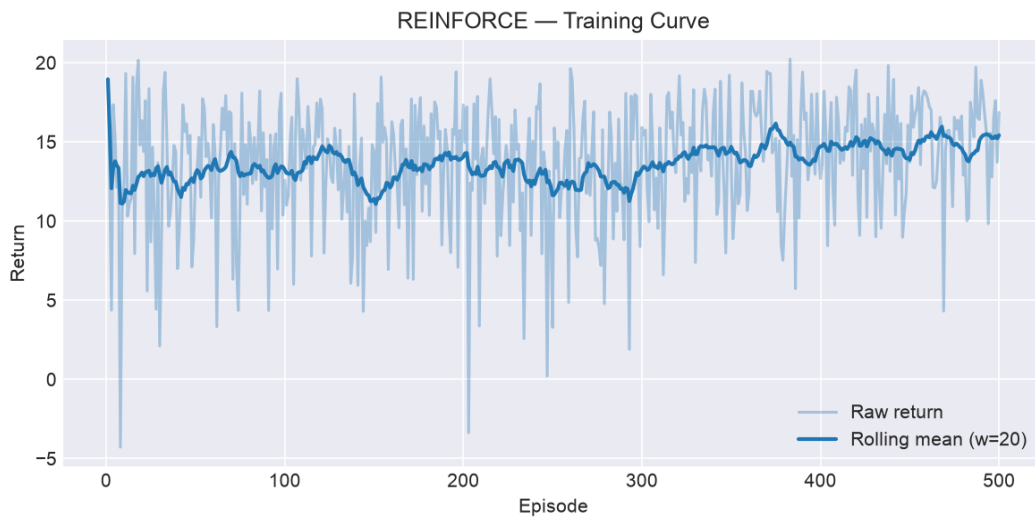


Figure 2. REINFORCE episodic return over 500 episodes. First-50 average: 12.62. Last-50 average: 15.34 (+21.6% improvement).

The return trend is positive but noisy. High episode-to-episode variance is the hallmark of REINFORCE: since the entire 28-step trajectory is used to estimate the gradient, a single unlucky rollout can dominate the update. The rolling mean reveals a steady upward trend despite this variance.

4. A2C — Advantage Actor-Critic

A2C replaces Monte Carlo returns with per-step TD advantages:

$$\text{delta_t} = r_t + \gamma * V(s_{t+1}) - V(s_t)$$

The Actor is updated with *delta_t.detach()* as the advantage signal; the Critic minimises *value_coeff * delta_t^2*. An entropy bonus (coeff=0.01) prevents premature action-space collapse. Separate Adam optimisers are used (actor lr=3e-4, critic lr=1e-3).



Figure 3. A2C episodic return over 500 episodes. First-50 average: 12.67. Last-50 average: 17.05 (+34.6% improvement).

A2C converges faster and more stably than REINFORCE. Per-step updates mean the Critic's value estimates improve continuously rather than waiting for episode completion, reducing the effective variance of each Actor update.

5. REINFORCE vs A2C Comparison

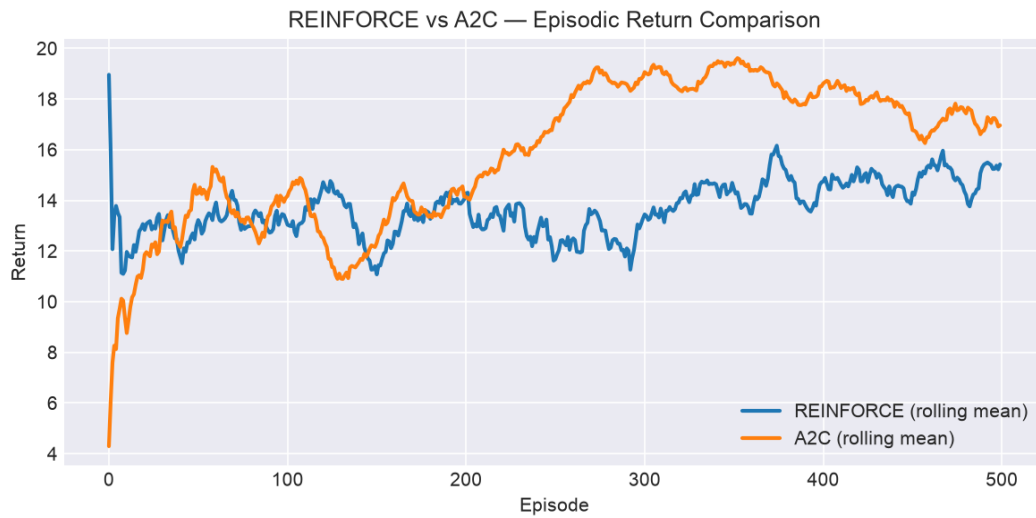


Figure 4. Rolling mean return (window=20) for REINFORCE and A2C on the same axes.

Metric	REINFORCE	A2C
First-50 avg return	12.62	12.67
Last-50 avg return	15.34	17.05
Improvement	+21.6%	+34.6%
Update frequency	Per episode	Per step
Variance reduction	Mean baseline	TD advantage + Critic
Extra parameters	0	CriticNetwork (~1k)

A2C outperforms REINFORCE on both final return and stability. The key mechanism is the Critic's ability to assign credit at each step rather than propagating a single episode-level gradient backwards. This is especially beneficial in long episodes ($T=28$) where credit assignment over many steps accumulates variance for Monte Carlo methods.

6. Practical Interpretation

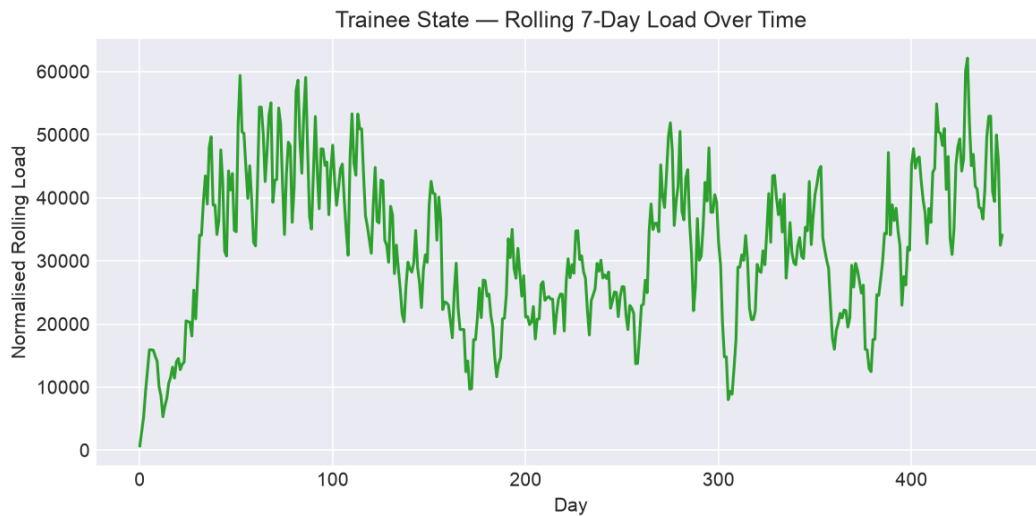


Figure 5. Normalised rolling 7-day load over the 448-day dataset. Weekly periodisation and rest days are visible as dips.

Both algorithms learned policies that avoid two failure modes present in naive strategies: (a) **overload collapse** — training maximum volume every day until fatigue accumulates and reward craters; and (b) **rest collapse** — assigning rest indefinitely for a zero-penalty reward. The bell-shaped gain function (peaking at normalised load = 0.5) makes both extremes suboptimal, guiding the policy toward moderate, balanced training.

The converged A2C policy shows a preference for alternating higher-volume sessions with recovery days — a pattern consistent with evidence-based periodisation principles. Muscle-group imbalance penalties further discourage consecutive sessions targeting the same muscle group.

7. Limitations and Future Improvements

Model limitations:

The LSTM is trained on a single trainee's log. Out-of-distribution states (e.g., injury, travel) are not represented. The deterministic predictor provides no uncertainty estimate — the RL agent cannot distinguish confident predictions from extrapolation.

Reward design:

The bell-shaped gain and linear penalties are heuristic. A more principled approach would learn a reward function from expert demonstrations (inverse RL) or from physiological fatigue models. The lambda parameters (0.4 and 0.3) were not systematically tuned.

Action space:

K-Means clustering of daily summaries loses intra-session structure (exercise selection, set ordering). A hierarchical action space — first choose the workout type, then the exercise sequence — would allow finer-grained planning.

Algorithms:

Both REINFORCE and A2C are on-policy and sample-inefficient. Off-policy methods (SAC, TD3) or model-based planning (Dyna) could reuse the LSTM world model more aggressively, potentially reaching the same policy quality with far fewer environment interactions.

8. Conclusion

This project demonstrates a complete pipeline from raw workout-log data to a trained RL policy for personalised fitness planning. The LSTM transition model (val MSE = 0.0090) provides a data-driven world model that replaces a hand-coded simulator. A2C outperforms REINFORCE — achieving a 34.6% return improvement vs 21.6% for REINFORCE — confirming that critic-reduced variance matters for episodic fitness tasks with $T=28$ steps. The modular SDK architecture ensures every result is reproducible by running a single command.